Received: 27 April 2018 | Revised: 7 January 2020 | Accepted: 20 January 2020

DOI: 10.1002/cpe.5693

SPECIAL ISSUE PAPER

WILEY

Performance evaluation of containers and virtual machines when running Cassandra workload concurrently

Sogand Shirinbab[1] | Lars Lundberg[1] | Emiliano Casalicchio[1,2]

[1]Department of Computer Science, Blekinge Institute of Technology, Karlskrona, Sweden

[2]Sapienza University of Rome, Rome, Italy

Correspondence
Lars Lundberg, Department of Computer Science, Blekinge Institute of Technology, Karlskrona, Sweden.
Email: lars.lundberg@bth.se

Summary

NoSQL distributed databases are often used as Big Data platforms. To provide efficient resource sharing and cost effectiveness, such distributed databases typically run concurrently on a virtualized infrastructure that could be implemented using hypervisor-based virtualization or container-based virtualization. Hypervisor-based virtualization is a mature technology but imposes overhead on CPU, networking, and disk. Recently, by sharing the operating system resources and simplifying the deployment of applications, container-based virtualization is getting more popular. This article presents a performance comparison between multiple instances of VMware VMs and Docker containers running concurrently. Our workload models a real-world Big Data Apache Cassandra application from Ericsson. As a baseline, we evaluated the performance of Cassandra when running on the nonvirtualized physical infrastructure. Our study shows that Docker has lower overhead compared with VMware; the performance on the container-based infrastructure was as good as on the nonvirtualized. Our performance evaluations also show that running multiple instances of a Cassandra database concurrently affected the performance of read and write operations differently; for both VMware and Docker, the maximum number of read operations was reduced when we ran several instances concurrently, whereas the maximum number of write operations increased when we ran instances concurrently.

KEYWORDS

Cassandra, cloud computing, containers, performance evaluation, virtual machine

1 | INTRODUCTION

Hypervisor-based virtualization began several decades ago and since then it has been widely used in cloud computing. Hypervisors, also called virtual machine monitors, share the hardware resources of a physical machine between multiple virtual machines (VMs). By virtualizing system resources such as CPUs, memory, and interrupts, it became possible to run multiple operating systems (OS) concurrently. The most commonly used hypervisors are kernel virtual machine (KVM), Xen Server, VMware, and Hyper-V. Hypervisor-based virtualization enables new features such as performance management, elastic resource scaling, and reliability services without requiring modifications to applications or operating systems. It also enables VM migration for load balancing and for consolidation to improve resource utilization and energy efficiency. However, hypervisor-level virtualization introduces performance overheads.[1-5] This overhead limits the use of hypervisor-level virtualization in performance critical domains.[6-8]

Recently, container-based virtualization has gained more popularity than hypervisor-based virtualization. A container is a lightweight operating system running inside the host system. An application running in a container has unshared access to a copy of the operating system. In other words, containers virtualize the operating system while hypervisors virtualize the hardware resources. Therefore, container-based virtualization is well known for providing savings in resource consumption and reducing the overhead of hypervisor-based virtualization while still providing isolation.[9,10] The main difference between the VM and the container architecture is that, for the VMs, each virtualized application includes an entire guest operating system and necessary binaries/libraries, while the container engine contains just the application and its dependencies (binaries/libraries).

Container-based virtualization has been offered by FreeBSD Jails (available since 2000) and Solaris Zones (available since 2004). In the beginning of 2008, a new Linux kernel was released in the form of Linux container (LXC).[11,12] Other alternatives to Linux-based containers are Open VZ[13,14] and Docker.[15-17] Recently, there have been several studies on performance of container-based virtualization technologies, especially Docker containers,[18,19] which are designed to run a single application per container, while LXC containers are more like VMs with a fully functional operating system.[15] The container-based architecture is rapidly becoming a popular development and deployment paradigm because of low overhead and portability. Disadvantages with containers are security issues (which will not be the focus in this study[20]), the limitations of not being able to run a different OS, and that the maturity level of management tools (eg, for live migration, snapshotting, and resizing) is lower than for VMs.

Since both containers and VMs have their set of benefits and drawbacks, one of the key points to select the proper virtualization technology for Big Data platforms is to assess the performance of hypervisor-based virtualized or container-based virtualized databases and how this relates to the performance without virtualization.[21] This article provides a detailed performance comparison running a distributed database on a number of physical servers, using VMware VMs, or using Docker containers. As database we selected Apache Cassandra[22-24] an open-source NoSQL distributed database widely adopted by companies using Docker and VMware and widely used in Big Data applications. Cassandra is known to manage some of the world's largest datasets on clusters with many thousands of nodes deployed across multiple data centers. Also, Cassandra Query Language (CQL) is user friendly and declarative.[25,26]

There are several NoSQL systems. In a previous study, the performance of Cassandra, using a large industrial application with similar properties to the one studied here, was compared with the performance of MongoDB, CounchDB, RethinkDB, and PostgreSQL.[27] In a similar study, the performance of Cassandra was compared with MongoDB and Riak.[28] The results from these studies show that Cassandra has the best performance; similar results, that also identify the performance benefits of Cassandra, have been reported in other studies*. Based on these and similar studies, Ericsson decided to use Cassandra in some of their Big Data telecom applications; since performance is the focus, it is logical to use the NoSQL system with the highest performance (ie, Cassandra) as the workload. Doing performance evaluations based on stress-tools and synthetic load is the most common approach (see Section 2.2 for details). In order to provide transparency and repeatability, we also use a stress tool. However, one contribution in our article is that the mix of read and write operations models a real Big Data enterprise Cassandra application from Ericsson.

Our results show that the container-based version had lower overhead compared with the VMware virtualized version. In fact, the performance of the container-based version was as good as the nonvirtualized version. Our experiments are conducted on servers with multicore CPUs. Although there are studies that demonstrate that relational databases,[29,30] and in memory, column based and key-value data stores[31,32] can benefit from GPUs to accelerate select like queries, we decided not to consider that possibility because, at the time we did the experiments, Cassandra did not have support for manycore architectures (GPU). Also, most server systems used for database applications in industry do not contain GPUs. The reason for this is that most industrial database applications, including the one we studied in this article, cannot benefit from the massive parallelism provided by GPUs.

This article significantly extends the work in a conference paper.[33] The main extension compared with the conference paper is that we have done additional measurements where several instances of the Cassandra database run concurrently on the cluster; the results in Figure 1 and in Figures 4 to 9 are new compared with the conference paper. We have also extended the related work section, improved the presentation, and provided a longer and more thorough discussion.

The presented work is organized as follows: In Section 2, we discuss related work. Section 3 describes the experimental setup and test cases. Section 4 presents the experimental results and we conclude our work in Section 5.

2 | RELATED WORK

Today, the container virtualization technology is gaining momentum.[34-36] While hypervisor-based virtualization (ie, virtual machine) is a mature technology,[1-5] which was introduced by IBM mainframes in the 1970s,[37] container-based technologies (eg, Docker and Linux Container [LXC]) have been introduced recently.[38] The major advantage of containers is that they achieve near-native performance.

From an analysis of the literature about performance evaluation of container runtime environments emerge that research works can be classified in studies about distributed databases workload and studies about other concurrent workloads.

*For example, https://www.datastax.com/products/compare/nosql-performance-benchmarks and https://www.scnsoft.com/blog/cassandra-performance

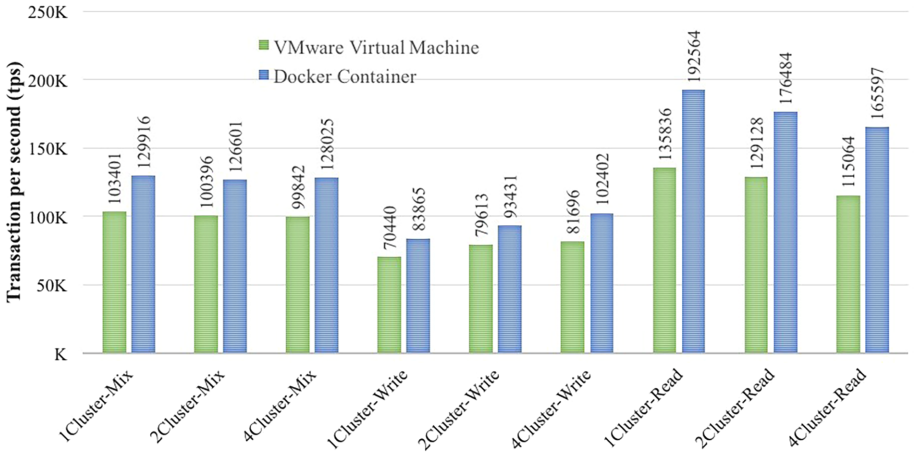

FIGURE 1 Transactions per second for VMware and Docker for different workloads and for different number of clusters (RF = 3).

2.1 | Distributed databases workload

Databases are often chosen to store and query large amounts of data. Traditionally, SQL databases were used in most data centers. However, because of scalability issues, NoSQL databases have gained popularity since 2007.[39] NoSQL databases support large volumes of data in a scalable and flexible manner. Currently, in the majority of data centers, online applications with NoSQL databases are hosted on VMs. Therefore, there is a need to evaluate NoSQL database performance in virtual environments. There are various studies trying to assess the performance impacts of virtualization on SQL and NoSQL databases.

In Reference 40, the authors compared the performance of three databases: a SQL database (PostgreSQL), and two NoSQL databases (MongoDB and Cassandra) using sensor data. They also compared running these databases on a physical machine and on a virtual machine. According to their results, virtualization has a huge effect on Cassandra read performance, while it has a moderate performance impact on MongoDB and increase the write performance on PostgreSQL. In Reference 41, the authors compared the performance of Docker container and KVM hypervisor using the MySQL database. According to their results, containers show equal or better performance than VMs in almost all cases. That is confirmed by our results. An in-memory distributed object store is considered as workload to assess the performance of VMware vSpere vs the native environment. In that case, the virtualization layer introduced an overhead of about 20% for read and 15% for write. In Reference 42, the authors compared the performance of two NoSQL databases (MongoDB and Cassandra) using different virtualization techniques, VMware (full virtualization) and Xen (paravirtualization). According to their results, VMware provides better resource utilization compared with Xen for NoSQL databases.

2.2 | Other concurrent workloads

There are several studies comparing hypervisor-based and container-based virtualization under different concurrent workloads.

In Reference 43, the authors assess the different measurement methodology for CPU and disk I/O intensive Docker workloads. While in Reference 44, the authors presented a preliminary study that correlates performance counters from the `/cgroup` file system with performance counters from the `/proc` file system for CPU intensive workloads.

In Reference 45, the authors compared the performance of the KVM and Xen hypervisors with Docker containers on the ARM architecture. According to their results, containers had better performance in CPU bound workloads and request/response networking, while hypervisors had better performance in disk I/O operations (because of the hypervisor's caching mechanisms) and TCP streaming benchmarks. Similarly, in References 46 and 47, the authors compared the performance of container-based virtualization with hypervisor-based virtualization for high performance computing (HPC). According to their result, container-based solutions delivered better performance than hypervisor-based solutions.

In Reference 48, the authors characterized the performance of three common hypervisors: KVM, Xen, and VMware ESXi and an open source container LXC across two generations of GPUs and two host microarchitectures, and across three sets of benchmarks. According to their results KVM achieved 98% to 100% of the base system's performance, while VMware and Xen achieved only 96% to 99%. In Reference 13, the authors compared the performance of an open source container technology OpenVZ and the Xen hypervisor. According to their results, container-based virtualization outperforms hypervisor-based virtualization. Their work is similar to our study. However, they considered wide-area motion imagery, full motion video, and text data, while in our case study, we were more interested in the performance of Cassandra databases.

In Reference 7, the authors compared the execution times of AutoDock3 (a scientific application) for Docker containers and VMs created using OpenStack. According to their results, the overall execution times for container-based virtualization systems are less than for hypervisor-based virtualization systems due to differences in start-up times. In Reference 9, the authors analyzed the process handling, file systems, and namespace isolation for container-based virtualization systems such as Docker, Linux Containers (LXC), OpenVZ, and Warden. From their assessment, containers have an advantage over VMs because of performance improvements and reduced start-up times. In Reference 8, the authors demonstrated that container-based systems are more suitable for usage scenarios that require high levels of isolation and efficiency such as HPC clusters. Their results indicate that container-based systems perform two times better for server-type workloads than hypervisor-based systems.

In Reference 49, the authors studied the performance of container platforms running on top the NeCTAR cloud infrastructure. Specifically, the authors compared the performance of Docker, Flockport (LXC) and VMs using the same benchmarks as in Reference 41. The comparison was intended to explore the performance of CPU, memory, network, and disk.

In Reference 50, the authors proposed a study on the interference among multiple applications sharing the same resources and running in Docker containers. The study focuses on I/O and it proposes also a modification of the Docker kernel to collect the maximum I/O bandwidth from the machine it is running on.

The performance of containers running on Internet-of-Things Raspberry Pi devices is investigated in Reference 51. As benchmarks are used: system benchmarks to independently stress CPU, memory, network I/O, disk I/O; and application benchmarks reproducing MySQL and Apache workload. The point of reference for comparison is the performance of the system without any virtualization.

In Reference 52, the authors investigate the impact of Docker configuration and resource interference on the performance and scalability of Spark-based Big Data applications.

In Reference 53, the authors present an extensive study of Docker storage, for wide range of file systems, and demonstrate its impact on system and workload performance. In Reference 38, the authors compared the performance of Docker when the Union file system and the CoW file system are used to build image layers.

2.3 | Main contribution in this article compared with existing literature

Our article differs from existing literature and advances the state of the art as follows. First, we evaluate the performance of Cassandra using the characteristics of a real enterprise Big Data telecom application from Ericsson. Second, our study assesses if and how much container-based platforms can speed up Cassandra vs a traditional VM deployment. Third, we run Cassandra on a distributed infrastructure rather than on a single server.

3 | EVALUATION

The goal of the experiment was to compare the performance of VMware VMs and Docker containers when running Cassandra. As a baseline for the comparison, we used the performance of Cassandra running on physical servers without any virtualization.

3.1 | Experimental setup

All our tests were performed on three HP servers DL380 G7 with a total of 16 processor cores (plus HyperThreading), 64 GB of RAM, and disk of size 400 GB. Cassandra 3.0.8 run on RHEL7 and this setup was identical for all test cases. The same version of Cassandra was used as well as for the load generators. VMware ESXi 6.0.0 was installed in case of virtualized workload using VMs. We used Docker version 1.12.6 in our container measurements. In our experiments, we consider the cases of 1, 2, and 4 Cassandra clusters concurrently running on the test-bed. Each Cassandra cluster consists of three Cassandra (virtual) nodes. Each virtual node is implemented either as a VM or as a container, and each of the three virtual nodes in a Cassandra cluster runs on different physical servers (there are three physical servers). Default settings are used to configure Cassandra and only the replication factor parameter is changed in the experiments.

3.2 | Workload

To generate workload, we used the Cassandra-stress tool.[38] The Cassandra-stress tool is a Java-based stress utility for basic benchmarking and load testing of a Cassandra cluster. Creating the best data model requires significant load testing and multiple iterations. The Cassandra-stress tool helps us by populating our cluster and supporting stress testing of arbitrary CQL tables and arbitrary queries on tables. The Cassandra package comes with a command-line stress tool (Cassandra-stress tool) to generate load on the cluster of servers, the `cqlsh` utility, a python-based command line

TABLE 1 Cassandra-stress tool sample commands

	Command
Populate the database	`cassandra-stress write n=40000000 -pop seq=1..40000000 -node -schema "replication(strategy=NetworkTopologyStrategy, datacenter1=3)"`
Mix-Load	`cassandra-stress mixed ratio\(write=1, read=3\) duration=30m -pop seq=1..40000000 -schema keyspace="keyspace1" -rate threads=100 limit=op/s –node`
Read-Load	`cassandra-stress read duration=30m -pop seq=1..40000000 -schema keyspace="keyspace1" -rate threads=100 limit=op/s –node`
Write-Load	`cassandra-stress write duration=30m -pop seq=1..40000000 -schema keyspace="keyspace1" -rate threads=100 limit=op/s –node`

client for executing CQL commands, and the `nodetool` utility for managing a cluster. These tools are used to stress the servers from the client and manage the data in the servers.

The Cassandra-stress tool creates a keyspace called keyspace1 and within that, tables named standard1 or counter1 in each of the nodes. These are automatically created the first time we run the stress test and are reused on subsequent runs unless we drop the keyspace using CQL. A write operation inserts data into the database and is executed prior to the load testing of the database. Later, after the data are inserted into the database, we run the mix workload, and then split up the mix workload and run a write only workload and a read only workload.

Below we described in detail each workload as well as the commands we used for generating the workloads:

- Mix-Load: To analyze the operation of a database while running both read and write operations during one operation, a mixed load command is used to populate the cluster. A mixed load consists of 75% read requests and 25% write requests generated for a duration of 30 minutes on a three-node Cassandra cluster. The command used for generating the mixed load is described in Table 1. In case of 2 or 4 Cassandra clusters, we have a separate workload generator for each Cassandra cluster. This mix of read and write operations is based on the characteristics of a Big Data Cassandra application developed by Ericsson.
- Read-Load: In addition to the mix workload, we measured the performance of the database for a read-only workload. In this case, 100% read requests are generated for a duration of 30 minutes on a three-node Cassandra cluster. The command used for generating the read-load is described in Table 1. In case of 2 or 4 Cassandra clusters, we have a separate workload generator for each cluster.
- Write-Load: In addition to the mix and read workloads, we measured the performance of the database for the write-only workload. In this case, 100% write requests are generated for duration of 30 minutes on a three-node Cassandra cluster. The command used for generating the write-load is described in Table 1. In case of 2 or 4 Cassandra clusters, we have a separate workload generator for each cluster.

Short descriptions of the Cassandra-stress tools input parameters (used in Table 1) is provided below:

- `mixed`: Interleave basic commands with configurable ratio and distribution. The cluster must first be populated by a write test. Here we selected a mixed load operation of 75% reads and 25% write.
- `write`: Multiple concurrent writes against the cluster.
- `read`: Multiple concurrent reads against the cluster.
- `n`: Specify the number of operations to run. Here we chose n = 40 000 000 to generate a 10 GB database.
- `pop`: Population distribution and intra-partition visit order. In this case we chose seq = 1… 40 000 000. This ensures that generated values do not overlap.
- `node`: To specify the address of the node to which data is to be populated.
- `schema`: Replication settings, compression, compaction, and so on. Here for the write operation we have modified the replication strategy to "NetworkTopologyStrategy" and set the number of replication from 1 to 3. Later for the mixed load we just set the name of the default keyspace which is "keyspace1".
- `duration`: It specifies the time in minutes to run the load.
- `rate`: Thread count, rate limit, or automatic mode (default is auto). In order to control the incoming traffic, we set the number of threads to 100. We have also limited the number of operations per second, so that we can measure the CPU utilization, write rate, and the latency mean for different number of transactions per seconds (tps) (40K, 80K, 120K, 160K, and 200K tps).

3.3 | Number of Cassandra clusters

There are three Cassandra nodes in each Cassandra cluster; one node on each physical machine. In case of two Cassandra clusters, there are two Cassandra nodes (belonging to different clusters) on each physical machine, and in case of four Cassandra clusters, there are four Cassandra nodes (belonging to different clusters) on each physical machine. Each Cassandra node is implemented as a VMware VM or as a Docker container, depending on the selected virtualization technology.

3.4 | Performance metrics

The performance of Docker, VMware, and the nonvirtualized solutions are measured using the following metrics:

- CPU utilization and
- mean latency of CSQ queries.

The CPU utilization is measured directly on the server nodes using `sar`. The latency is measured on the client side, by the stress test tool.

3.5 | Test cases

As mentioned before, three different deployments of Cassandra clusters are considered:

- *Cassandra-Non-Virtualized*: In this case, three servers are allocated and on each server we run a Cassandra node. All servers are connected using a high-speed isolated LAN and create a three-node Cassandra cluster.
- *Cassandra-VM*: In this case, one VMware virtual machine per Cassandra node is instantiated on each host, that is, for the case with four Cassandra clusters, there are in total 12 VMware VMs. Each Cassandra cluster is spread out on the three physical servers.
- *Cassandra-Docker*: In this case, a version of Cassandra using Docker container is created and one container per Cassandra node is deployed on each server, that is, for the case with four Cassandra clusters, there are in total 12 Docker containers. Each Cassandra cluster is spread out on the three physical servers.

In each test case, we first experiment with different Cassandra workload scenarios, that is, the Mix, Read, and Write workloads. Second, for the case with one Cassandra cluster, we experiment with various replication-factor configurations for the Cassandra cluster. In Cassandra, the input splits (ie, a set of table rows) are replicated among the nodes based on a user-set replication factor (RF). This design prevents data loss and helps with fault tolerance in case of node failure.[54] In our experiments, we investigate three different replication-factor settings: RF = 1, RF = 2, and RF = 3. In our test environment with three Cassandra node clusters, replication factor three means that each node should have a copy of the input data splits. For the case of RF = 2, the seed node decides, usually based on a random allocation algorithm, where to store replicas. In the case of RF = 1, each node only receives a portion of the input data (ie, of the table rows) and no replicas are created. In the case of 2 or 4 Cassandra clusters, we show results only for replication factor RF = 3, that is, the most critical case, as explained in what follows.

4 | EXPERIMENTAL RESULTS

Figure 1 gives an overview of our results. The figure shows the maximum number of tps on the same hardware for different numbers of Cassandra clusters implemented as VMs or containers, given a replication factor RF = 3. The figure shows that the performance of Docker containers is in general 30% higher than the performance of VMs, and, as expected, that the performance for the read workload is higher than the performance for the write workload. However, the figure shows that the effect of having multiple Cassandra clusters is different for the write and the read workload; writing benefits from having multiple Cassandra clusters, whereas the read performance is highest for one cluster. There are two effects that affect the performance in the case of multiple Cassandra clusters: first the overhead increases when the number of clusters increases for both the container and the VM case (see Figures 4 to 6 for details); this is the reason why the performance for the read workload decreases when the number of clusters increases. Second, the VMware hypervisor and the Docker system use affinity scheduling in the sense that a VM or a container tends to be scheduled on a certain subset of the processor cores. As a consequence, the cache hit ratio increases in case of multiple Cassandra clusters. For the read workload, data can be freely copied to all processor caches, so affinity scheduling has no, or little, effect on the cache hit ratio. For the write workload, the copies on other caches need to be invalidated when a data item is written, that is, for this case, the increased cache hit

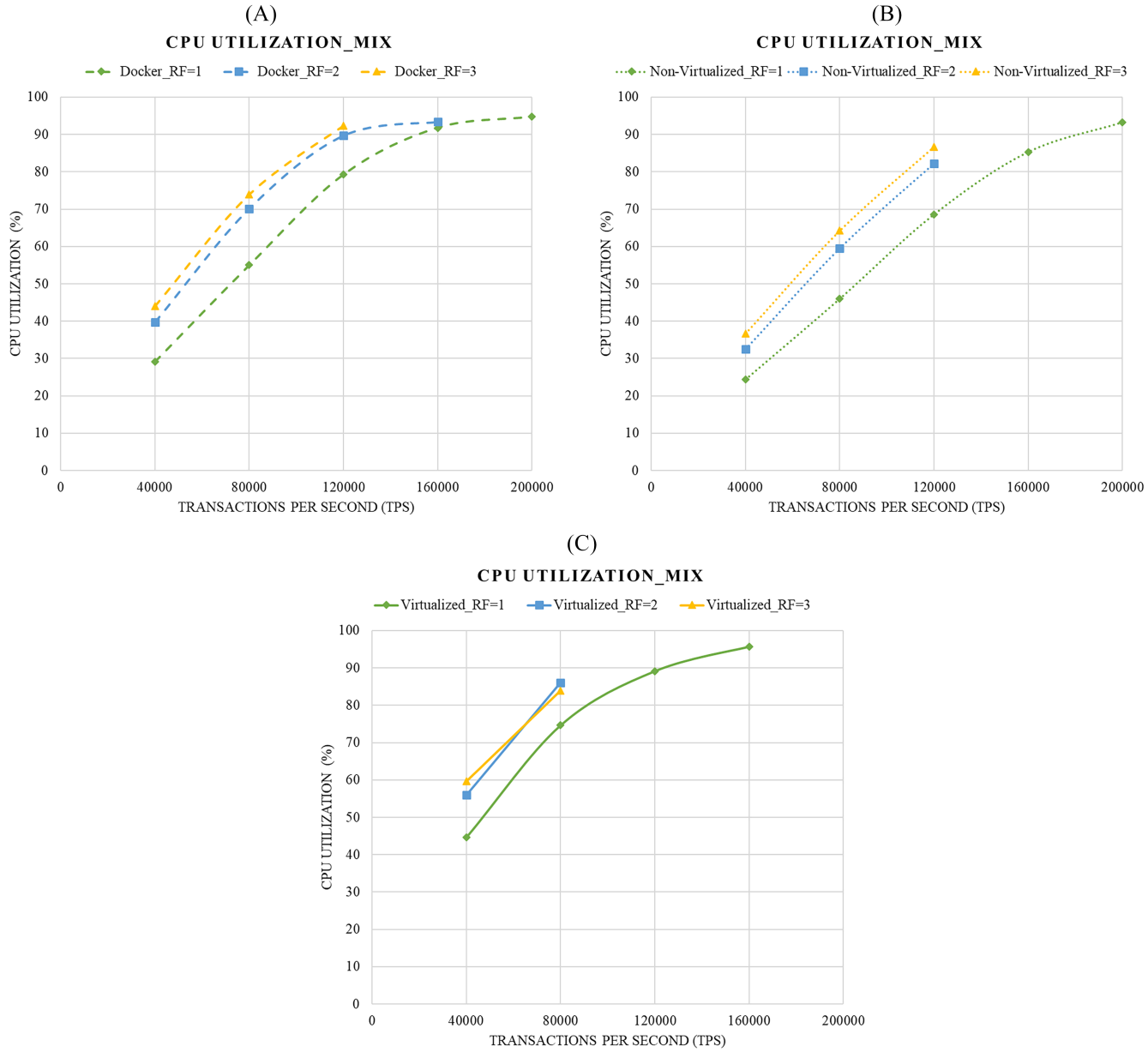

FIGURE 2 CPU utilization for the mixed workload for Docker (A), VMware (C), and nonvirtulized (B).

ratio due to affinity scheduling becomes important. This is the reason why the performance for the write workload increases when the number of clusters increases.

In the remaining of this section, we will look at more detailed result

4.1 | One Cassandra cluster

Figure 2 shows the results of running one three-node Cassandra cluster in a nonvirtualized environment, a VM environment, or a container environment while having a mix load. The results show that both the nonvirtualized and the container environments with RF = 1 can handle a maximum of 200K (tps). The VM environment with RF = 1 can only handle 160K (tps). For the VM case, the CPU overhead is up to 29% higher compared with the nonvirtualized version. One reason for getting very high overhead could be the additional layers of processing required by the virtualization software, that is, VMware. Moreover, the container case work completely in memory, which makes it possible to cope with the small overhead introduced by containers and to reach the same performance as the nonvirtualized case. This feature of containers is fully exploited in

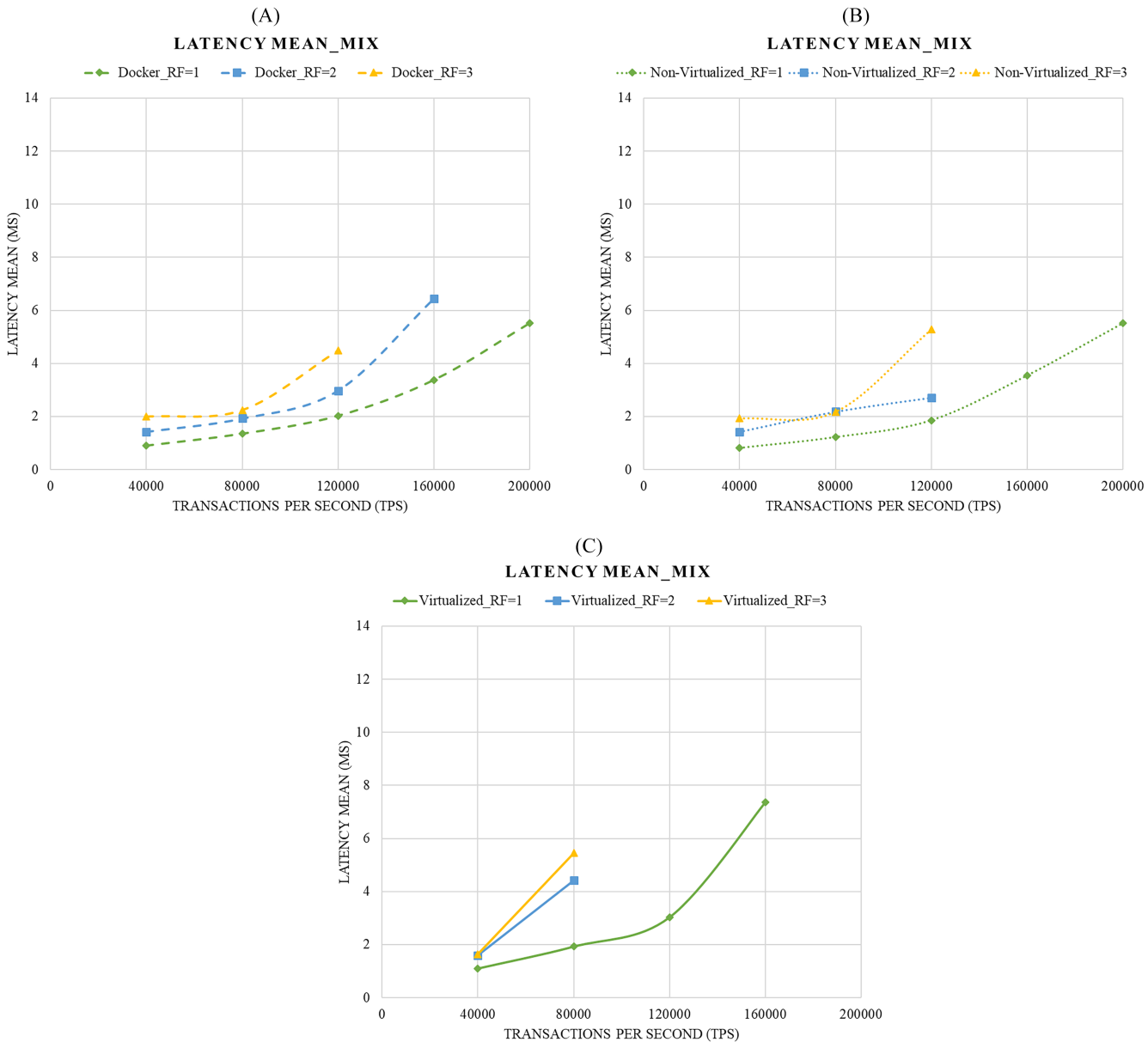

FIGURE 3 Mean latency for the mixed workload for Docker (A), VMware (C), and nonvirtulized (B).

the case of RF = 2 and RF = 3 when the container version significantly outperforms the VM version and has a throughput equal or higher than the nonvirtualized case.

The effect of full in-memory access to the data is evident also if we observe the latency. For RF = 1, the nonvirtualized and the container versions have the same latency, which is much lower than the latency for the VM case. For RF = 2, the container case can serve a higher number of transaction than the other two cases; and for RF = 3, while the number of tps is the same for the nonvirtualized and the container cases, the latter shows a somewhat lower latency.

In general, RF = 1 is always performing better than RF = 2 and RF = 3. One reason is that when the RF in set to one, only one copy of data is written and this process is very fast in Cassandra. Figure 3 shows that the latency mean value for RF = 1 is lower than for RF = 2 and RF = 3. Although RF = 1 is fast, from a high availability point of view, the data center providers prefer RF = 2 or RF = 3. Cassandra is using different consistency levels in order to wait for the respond from several nodes, and in our case, we set the consistency level to be Quorum. Quorum means that Cassandra returns the record after a quorum of replicas have responded.

Table 2 shows a summary of the CPU utilizations and mean latencies for the case with one Cassandra cluster. The table shows that for all workloads and replication factors, the CPU utilization and the latency were lowest for the nonvirtualized case, and that VMware has the largest CPU utilization and also the highest latency.

TABLE 2 Summary of the performance results for one cassandra cluster

		Docker		Nonvirtualized		VMware	
RF	Workload	CPU Util (%)	Latency (ms)	CPU Util (%)	Latency (ms)	CPU Util (%)	Latency (ms)
1	Mix (80K tps)	55	1.35	46	1.2	75	1.9
	Read (60K tps)	39	0.9	32	0.9	57	1.2
	Write (20K tps)	14	0.7	12	0.6	24	0.8
2	Mix (80K tps)	70	1.9	59	2.1	86	4.4
	Read (60K tps)	50	1.5	41	1.4	65	1.6
	Write (20K tps)	22	0.7	18	0.7	36	0.9
3	Mix (80K tps)	74	2.2	64	2.1	84	5.45
	Read (60K tps)	54	1.9	45	1.9	68	1.6
	Write (20K tps)	26	0.8	22	0.8	41	1

Abbreviation: tps, transactions per seconds.

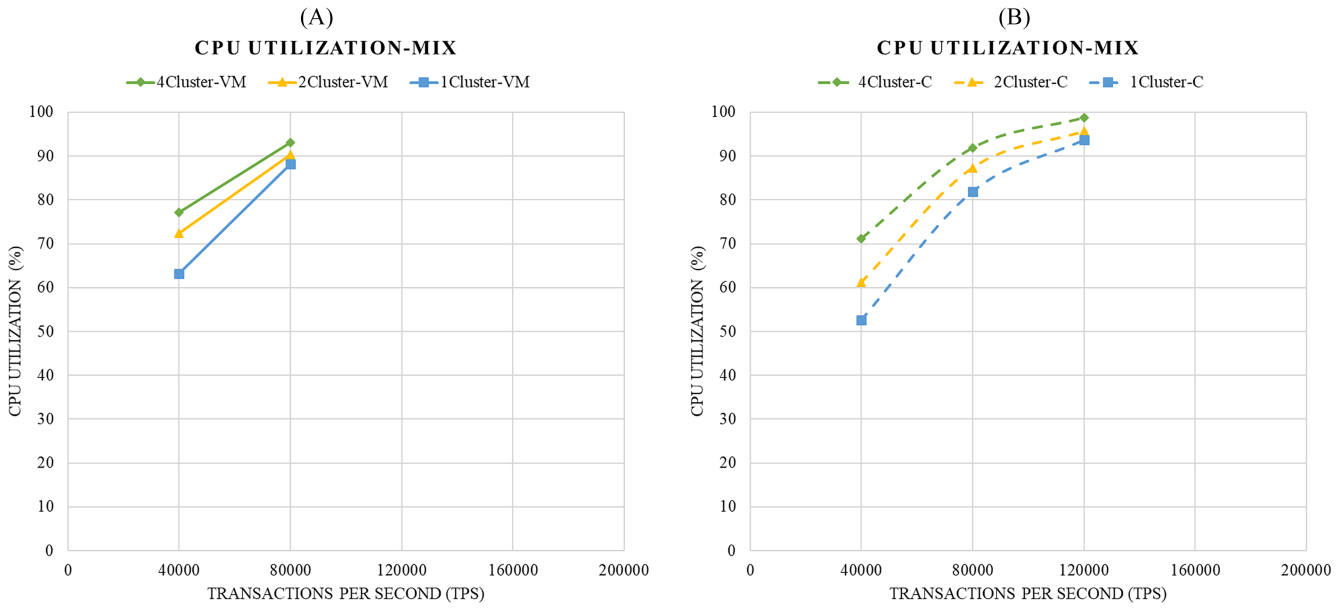

FIGURE 4 CPU utilization for the mixed workload for VMs (A) and containers (B) for different cluster sizes.

4.2 | Multiple Cassandra clusters

In the following set of experiments, we consider the case of deploying an increasing number of Cassandra clusters on the test bed, specifically: 1, 2, and 4 clusters.

We first make the general observation that for all Cassandra workloads and for both the container and the VM case, the CPU utilization increases when the number of Cassandra clusters increases (see Figures 4-6). Similarly, for all Cassandra workloads and for both the container and the VM case, the mean latency increases when the number of Cassandra clusters increases (see Figures 7-9). The workload specific comments are provided below.

Figure 4A shows the CPU utilization for the mix workload for Cassandra clusters deployed with VMware VMs. Figure 4B shows the CPU utilization for the mixed workload for Cassandra clusters deployed using Docker containers. As shown in Figure 1, the maximum number of tps for the mix workload is approximately 30% higher for containers compared with VMs. Figure 4 shows that for a certain number of tps, the CPU utilization is a bit higher when using VMs compared with containers. The difference in CPU utilization for VMs and containers is, however, not very significant.

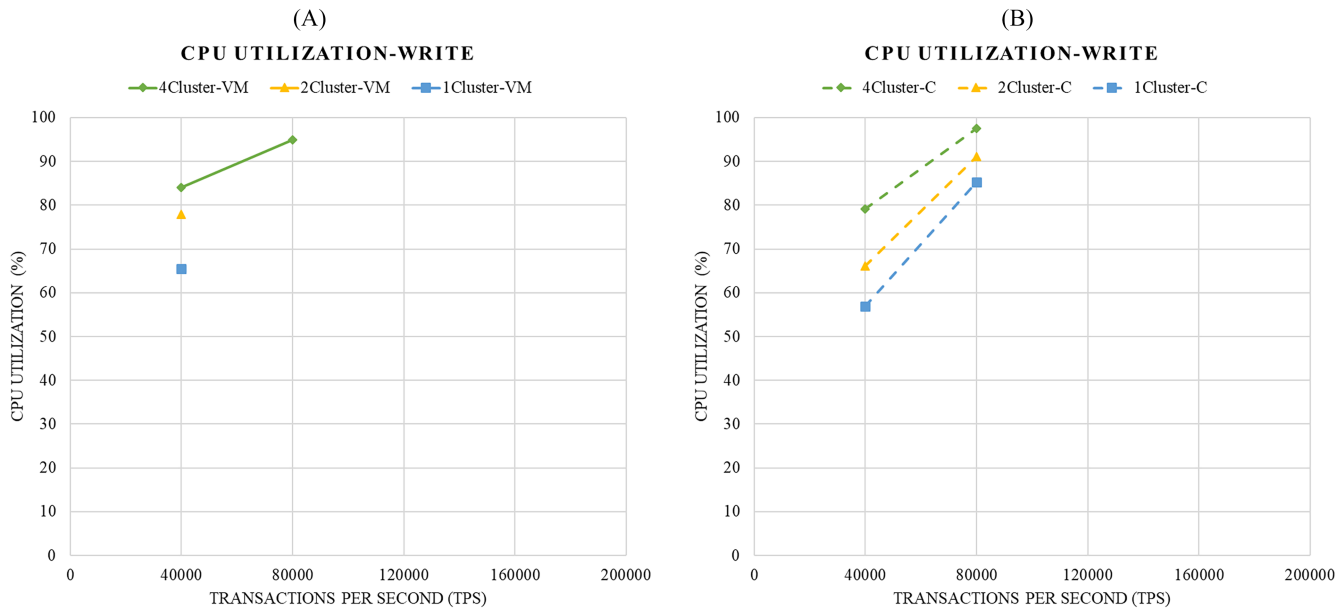

FIGURE 5 CPU utilization for the write workload for VMs (A) and containers (B) for different cluster sizes.

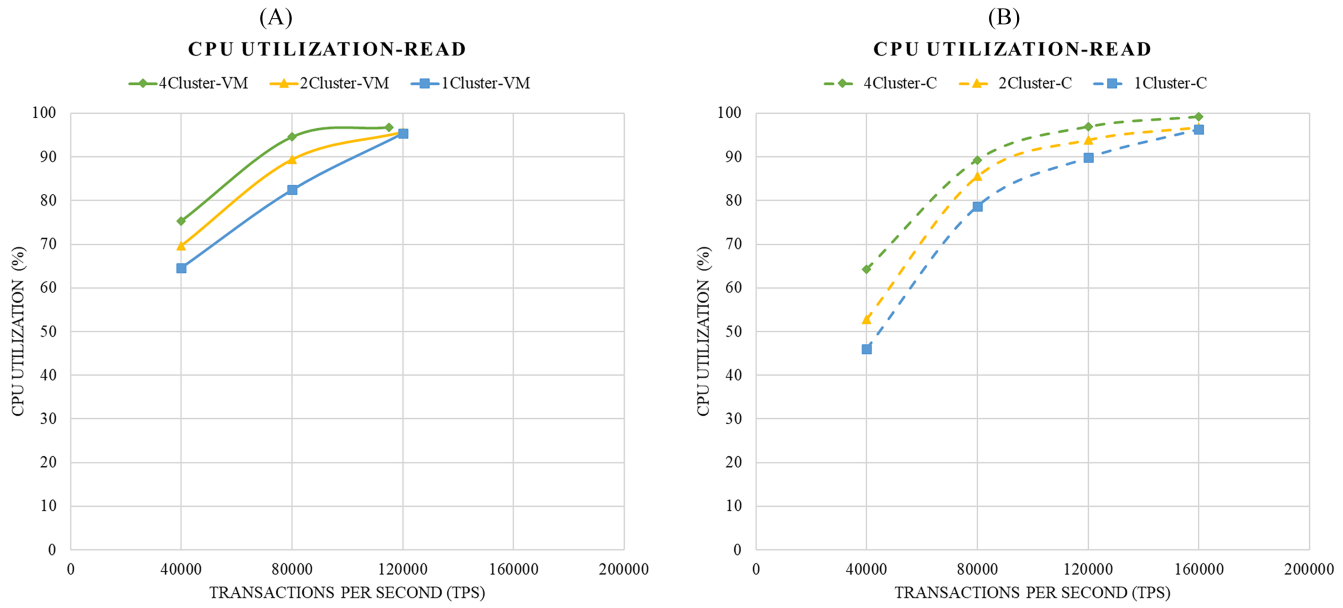

FIGURE 6 CPU utilization for the read workload for VMs (A) and containers (B) for different cluster sizes.

Figure 5A shows the CPU utilization for the write workload for Cassandra clusters deployed with VMware VMs. Figure 5B shows the CPU utilization for the write workload for Cassandra clusters deployed using Docker containers. As shown in Figure 1, the maximum number of tps for the write workload is 20% to 25% higher for containers compared with VMs. Figure 5 shows that for a certain number of tps, the CPU utilization is a bit higher when using VMs compared with containers. The difference in CPU utilization for VMs and containers is, however, not very significant.

Figure 6A shows the CPU utilization for the read workload for Cassandra clusters deployed with VMware VMs. Figure 6B shows the CPU utilization for the read workload for Cassandra clusters deployed using Docker containers. As shown in Figure 1, the maximum number of tps for the read workload is more than 40% higher for containers compared with VMs. Figure 6 shows that for a certain number of tps, the CPU utilization is higher when using VMs compared with containers.

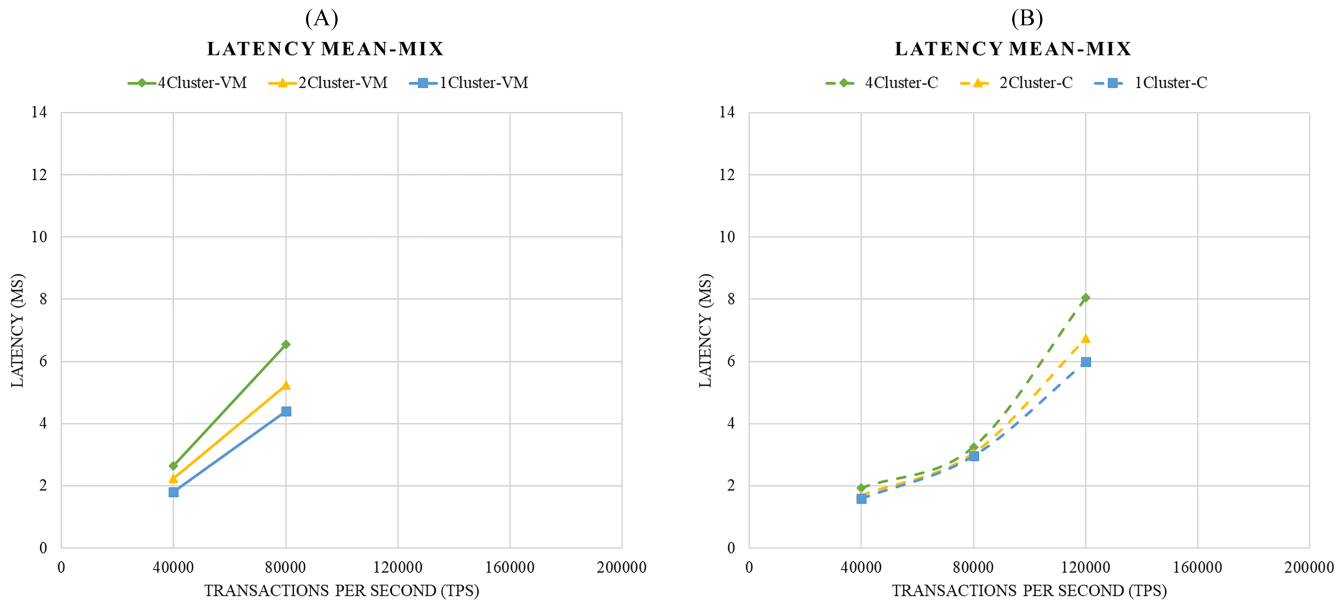

FIGURE 7 Mean latency for the mixed workload for VMs (A) and containers (B) for different cluster sizes.

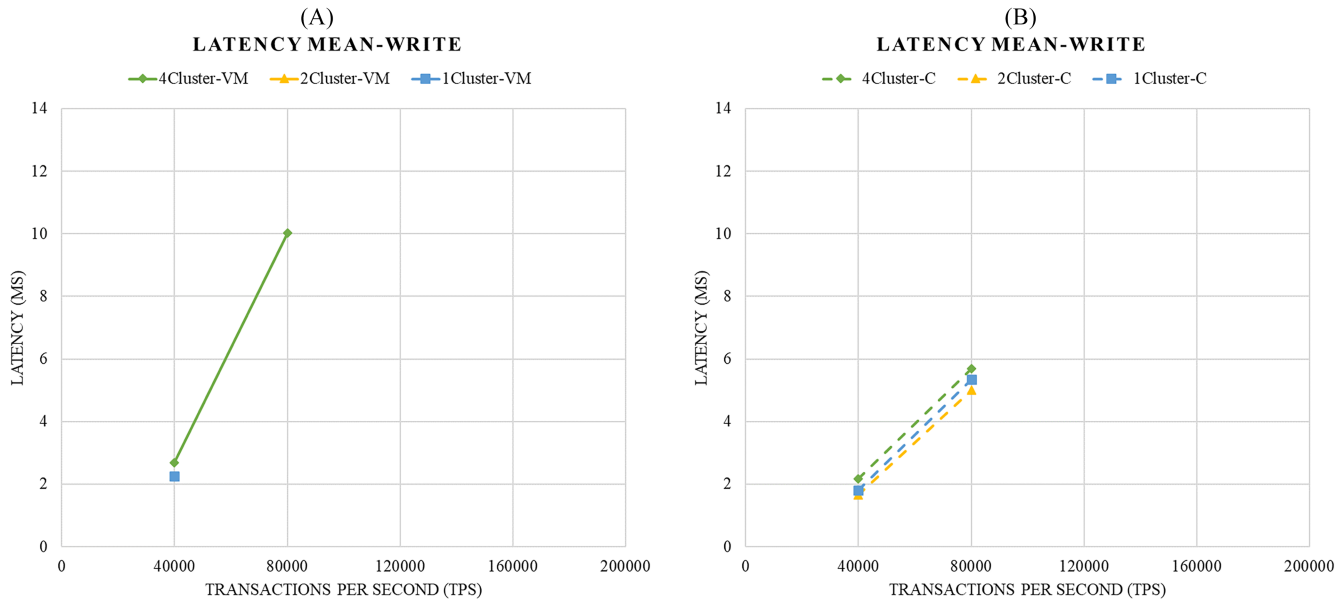

FIGURE 8 Mean latency for the write workload for VMs (A) and containers (B) for different cluster sizes.

Figure 7A shows the mean latency for the mix workload for Cassandra clusters deployed with VMware VMs. Figure 7B shows the mean latency for the mix workload for Cassandra clusters deployed using Docker containers. Figure 7 shows that for a certain number of tps, the latency is higher when using VMs compared with containers.

Figure 8A shows the mean latency for the write workload for Cassandra clusters deployed with VMware VMs. Figure 8B shows the mean latency for the write workload for Cassandra clusters deployed using Docker containers. Figure 8 shows that for a certain number of tps, the latency is higher when using VMs compared with containers, particularly when the number of transactions is close to maximum.

Figure 9A shows the mean latency for the read workload for Cassandra clusters deployed with VMware VMs. Figure 9B shows the mean latency for the read workload for Cassandra clusters deployed using Docker containers. Figure 9 shows that for a certain number of tps, the latency is higher when using VMs compared with containers, particularly when the number of transactions is close to maximum.

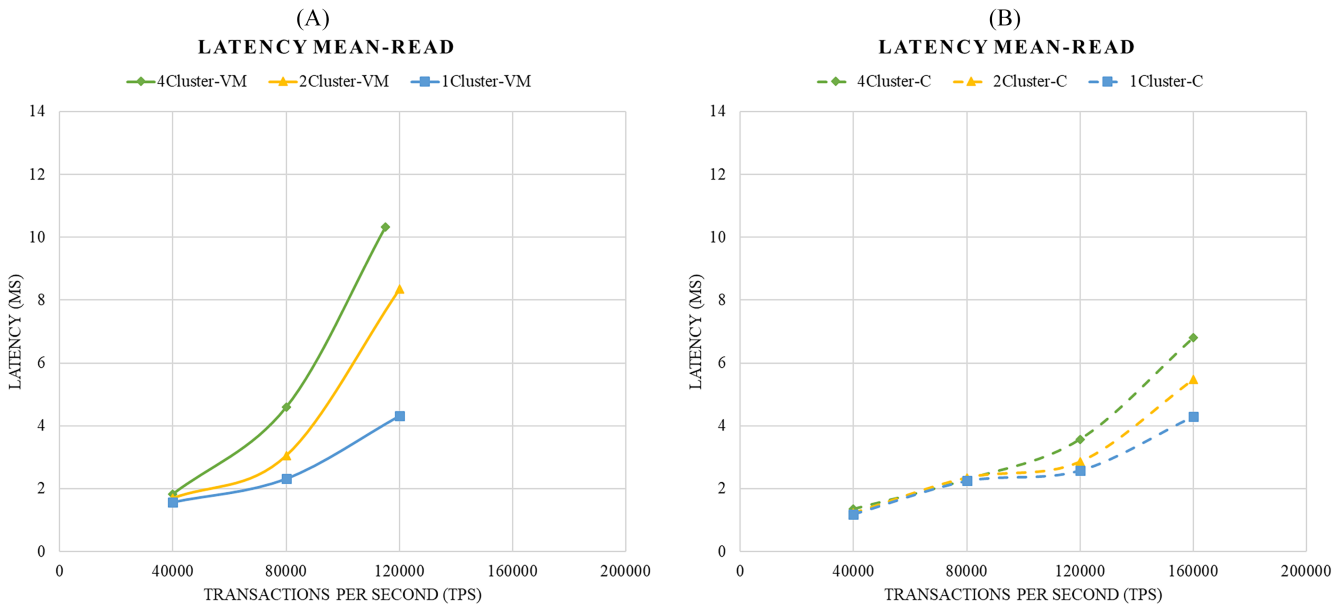

FIGURE 9 Mean latency for the read workload for VMs (A) and containers (B) for different cluster sizes.

5 | CONCLUSIONS AND FUTURE WORK

In this article, we investigate which solution is better for a distributed Cassandra database: nonvirtualized, virtualized (VMware) or Docker? The overall result showed that the biggest issue with running the virtualized VMware solution is the significant resource and operational overheads that affect the performance of the application. The Docker solution addresses the challenges of virtualization by packaging the applications and their dependencies into lightweight containers. According to our results, the Docker solution consumed fewer resources and operational overheads compared with the virtualized VMware solution. Also, the performance in terms of maximum number of tps was at least as high for the Docker solution as for the nonvirtualized case. As discussed in Sections 1 and 2, containers have often shown to be more efficient than VMs. Our study verifies and quantifies this effect for our real-world Cassandra workload.

We also investigated the effect of dividing a Cassandra database into multiple independent clusters. It turned out that write operations benefit from having multiple Cassandra clusters, whereas the read performance is highest for one cluster. The reason for this is that there are two aspects that affect the performance in the case of multiple clusters: first, the overhead increases when the number of clusters increases for both the container and the VM case. Second, the VMware hypervisor and the Docker system use affinity scheduling in the sense that a VM or a Docker container tends to be scheduled on a certain subset of the processor cores, which increases the cache hit ratio in case of multiple Cassandra clusters. For the read workload, affinity scheduling has no, or little, effect on the cache hit ratio and performance. However, for the write workload, affinity scheduling increases the cache hit ratio and performance.

Even though the Docker container solution is showing very low overhead and system resource consumption, it suffers from security problems when storing data, which is crucial for database protection. Comparing containers with VMs, containers cannot be secure candidates for databases because all containers share the same kernel and are therefore less isolated than VMs. A bug in the kernel affects every container and results in significant data loss. On the other hand, hypervisor-based virtualization is a mature and (relatively) secure technology.

According to our results, hypervisor-based virtualization suffers from noticeable overhead, which effects the performance of the databases. A first recommendation, valid for deployments in a single tenant environment, is to use containers directly on the host, without any other virtualization level in the middle. Orchestration frameworks such as Kubernetes could be used to implement automation, high availability, and elasticity (like in any other classical cloud setting). As a second advice, since both containers and VMs have their set of benefits and drawbacks, an alternative solution could be to combine the two technologies. That would increase isolation, important in multitenancy environments, and allow us to benefit from the full in-memory features of containers. In the future, we plan to investigate such alternative solutions by running containers inside VMs running Cassandra workload. In this way, we may get the benefits of both the security of the virtual machine and the execution speed of containers.

ACKNOWLEDGMENTS

This work is funded by the research project Scalable resource-efficient systems for Big Data analytics financed by the Knowledge Foundation (grant: 20140032) in Sweden

ORCID

Lars Lundberg https://orcid.org/0000-0002-5031-8215
Emiliano Casalicchio https://orcid.org/0000-0002-3118-5058

REFERENCES

1. Li J, Wang Q, Jayasinghe D, Park J, Zhu T, Pu C. Performance overhead among three hypervisors: an experimental study using hadoop benchmarks. *IEEE Int Congress Big Data*. 2013;9-16.
2. Soriga SG, Barbulescu M. A comparison of the performance and scalability of Xen and KVM hypervisors. *RoEduNet*. 2013:1-6.
3. Hwang J, Zeng S, Wu FY, Wood T. A component-based performance comparison of four hypervisors. *IFIP/IM*. 2013:269-276.
4. Che J, He Q, Gao Q, Huang D. Performance measuring and comparing of virtual machine monitors. *EUC*. 2008:381-386.
5. Shirinbab S, Lundberg L, Ilie D. Performance comparison of KVM, VMware and XenServer using a large telecommunication application. Paper presented at: 5th International Conference on Cloud Computing, GRIDs, and Virtualization; 2014:114-122.
6. Yu L, Chen L, Cai Z, Shen H, Liang Y, Pan Y. Stochastic load balancing for virtual resource management in datacenters. *IEEE Trans Cloud Comput*. 2016;4:1-14.
7. Adufu T, Choi J, Kim Y. Is container-based technology a winner for high performance scientific applications? *IEICE*. 2015;507-510.
8. Soltesz S, Pötzl H, Fiuczynski M, Bavier A, Peterson L. Container-based operating system virtualization: a scalable, high-performance alternative to hypervisors. *EuroSys*. 2007:275-287.
9. Dua R, Raja AR, Kakadia D. Virtualization vs containerization to support PaaS. *IC2E*. 2014:610-614.
10. Adufu T, Choi J, Kim Y. Is container-based technology a winner for high performance scientific applications? *APNOMS*. 2015:507-510.
11. LXC. Linux Containers. 2016. https://linuxcontainers.org/
12. Pahl C. Containerization and the PaaS cloud. *IEEE Cloud Comput*. 2015;2(3):24-31.
13. Wu R, Deng A, Chen Y, Blasch E, Liu B. Cloud technology applications for area surveillance. *NAECON*. 2015:89-94.
14. OpenVZ. *OpenVZ Virtuozzo Containers*; 2016. https://openvz.org/
15. Ismail BI, Mostajeran Goortani E, Bazli Ab Karim M, et al. Evaluation of Docker as Edge computing platform. *ICOS*; 2015:130-135.
16. Docker. *Docker*. 2016. https://www.docker.com/
17. Anderson C. Docker [software engineering]. *IEEE Softw*. 2015;32(3):102-c3.
18. Kan C. DoCloud: an elastic cloud platform for web applications based on Docker. *ICACT*; 2016:478-483.
19. Preeth EN, Mulerickal FJP, Paul B, Sastri Y. Evaluation of Docker containers based on hardware utilization. *ICCC*. 2015:697-700.
20. Gkortzis A, Rizou S, Spinellis D. An empirical analysis of vulnerabilities in virtualization technologies. Paper presented at: Cloud Computing Technology and Science Conference; 2017:533-538.
21. Okur MC, Buyukkececi M. Big data challenges in information engineering curriculum. *EAEEIE*, 2014:1-4.
22. Jain R, Iyengar S, Arora A. Overview of popular graph databases. *ICCCNT*, 2013:1-6.
23. Cassandra. *Cassandra*. 2016. http://cassandra.apache.org/
24. Carpenter J, Hewitt E. Cassandra: The Definitive Guide. O'Reilly Media, Incorporated, Sebastopol; 2016.
25. Lourenco JR, Abramova V, Cabral B, Bernardino J, Carreiro P, Vieira M. No SQL in practice: a write-heavy enterprise application. *IEEE Int Congress Big Data*. 2015;584-591.
26. Chebotko A, Kashlev A, Lu S. A big data modeling methodology for apache cassandra. *IEEE Int Congress Big Data*. 2015;238-245.
27. Niyizamwiyitira C, Lundberg L. Performance evaluation of SQL and NoSQL database management systems in a cluster. *Int J Database Manag Syst*. 2017;9(6):1-24.
28. Klein J, Gorton I, Ernst N, Donohoe P, Pham K, Matser C. Performance Evaluation of NoSQL databases: a case study. *PABS'15*. 2015:5. doi:https://doi.org/10.1145/2694730.2694731.
29. Bakkum P, Skadron K. Accelerating SQL database operations on a GPU with CUDA. *GPGPU-3*. 2010:94-103.
30. Breß S, Heimel M, Siegmund N, Bellatreche L, Saake G. GPU-accelerated database systems: survey and open challenges. In: Hameurlain A, Küng J, Wagner R, et al., eds. *Transactions on Large-Scale Data- and Knowledge-Centered Systems XV. Lecture Notes in Computer Science*. Vol 8920. Berlin, Germany: Springer; 2014:1-35.
31. Breß S. The design and implementation of CoGaDB: a column-oriented GPU-accelerated DBMS. *Datenbank-Spektrum*. 2014;14(3):199-209.
32. Zhao D, Wang K, Qiao K, Li T, Sadooghi I, Raicu I. Toward high-performance key-value stores through GPU encoding and locality-aware encoding. *J Parallel Distrib Comput*. 2016;96(C):27-37.
33. Shirinbab S, Lundberg L, Casalicchio E. Performance evaluation of container and virtual machine running cassandra workload. *CloudTech*. 2017:1-8.
34. Liu D, Zhao L. The research and implementation of cloud computing platform based on Docker. *ICCWAMTIP*. 2014:475-478.
35. Bernstein D. Containers and Cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Comput*. 2014;1(3):81-84.
36. Naik N. Migrating from virtualization to dockerization in the cloud simulation and evaluation of distributed systems. *MESOCA*. 2016;1-8.
37. Goldberg RP. Survey of virtual machine research. *Computer*. 1974;7(6):34-45.
38. Dua R, Kohli V, Patil S, Patil S.Performance analysis of Union and CoW File Systems with Docker. *CAST*. 2016:550-555.
39. Leavitt N. Will NoSQL databases live up to their promise? *Computer*. 2010;43(2):12-14.
40. van der Veen JS, van der Waaij B, Meijer RJ Sensor data storage performance: SQL or NoSQL, physical or virtual. Paper presented at: IEEE Fifth International Conference on Cloud Computing; 2012:431-438.
41. Felter W, Ferreira A, Rajamony R, Rubio J. An updated performance comparison of virtual machines and Linux containers. *ISPASS*; 2015:171-172.
42. Martins G, Bezerra P, Gomes R, Albuquerque F, Costa A. Evaluating performance degradation in NoSQL databases generated by virtualization. *LANOMS*; 2015:84-91.
43. Morabito R, Kjallman J, Komu M. Hypervisors vs. lightweight virtualization: a performance comparison. *IC2E*. 2015:386-393.
44. Casalicchio E, Perciballi V. Auto-scaling of containers: the impact of relative and absolute metrics. *FAS*W*. 2017:207-214.
45. Raho M, Spyridakis A, Paolino M, Raho D. KVM, Xen and Docker: a performance analysis for ARM based NFV and cloud computing. *AIEEE*. 2015;1-8.

46. Zhang J, Lu X, Panda DK. Performance characterization of hypervisor- and container-based virtualization for HPC on SR-IOV enabled infiniband clusters. *IPDPSW*. 2016:1777-1784.
47. Chung MT, Quang-Hung N, Nguyen M, Thoai N. Using Docker in high performance computing applications. *ICCE*. 2016:52-57.
48. Walters JP, Younge AJ, Kang DI, Yao KT, Kang M, Crago SP, Fox GC. GPU Passthrough performance: a comparison of KVM, Xen, VMWare ESXi, and LXC for CUDA and OpenCL Applications. Paper presented at: IEEE 7th International Conference on Cloud Computing; 2014:636-643.
49. Kozhirbayev Z, Sinnott RO. A performance comparison of container-based technologies for the Cloud. *Future Generation Comput Syst*. 2017;68:175-182.
50. McDaniel S, Herbein S, Taufer M. A two-tiered approach to I/O quality of service in Docker containers. Paper presented at: IEEE International Conference on Cluster Computing; 2015:490-491.
51. Morabito R. A performance evaluation of container technologies on Internet of Things devices. Paper presented at: IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS); 2016:999-1000.
52. Ye K, Ji Y. Performance tuning and modeling for big data applications in Docker containers. *NAS*. 2017:1-6.
53. Tarasov V, Rupprecht L, Skourtis D, et al. In Search of the ideal storage configuration for Docker containers. *FAS*W*. 2017:199-206.
54. Dede E, Sendir B, Kuzlu P, Hartog J, Govindaraju M. An evaluation of Cassandra for hadoop. Paper presented at: IEEE Sixth International Conference on Cloud Computing; 2013:494-501.

How to cite this article: Shirinbab S, Lundberg L, Casalicchio E. Performance evaluation of containers and virtual machines when running Cassandra workload concurrently. *Concurrency Computat Pract Exper*. 2020;32:e5693. https://doi.org/10.1002/cpe.5693